PRODUCT REQUIREMENTS DOCUMENT

TaskFlow API

A secure, rate-limited, versioned REST API for task and project management

DOCUMENT OWNER	Satyam Pandey
STATUS	v1.0.0, Released
LAST UPDATED	July 2026
REPOSITORY	github.com/SatyamPandey07/REST-API-with-Rate-Limiting-Docs

FastAPI PostgreSQL JWT Auth Rate Limiting API Versioning 99% Test Coverage

Table of Contents

01 Executive Summary

TaskFlow API is a backend service for managing projects and tasks. On the surface, that sounds simple: create a project, add some tasks, mark them done. What makes this project worth documenting is everything underneath that surface. It handles who is allowed to see what, how much traffic any one user can send before something breaks, how the API can change in the future without breaking people already using it, and how someone with zero coding background can still open it up and use it.

In short, this is what a small but seriously built API looks like when it's designed the way a real product team would design it, not just enough to make a demo work. This document walks through what the product does, who it's for, why certain decisions were made instead of the obvious alternative, and what's still left to do. It's written so that a developer, a hiring manager, or someone with no technical background at all can read it and come away understanding the product.

02 The Problem This Solves

Most small API projects stop at "it works." You can create a task, read it back, and that's the demo. The problem is that almost every real API eventually runs into the same five issues, and how a team handles them says a lot about whether the product is ready for real use.

THE ISSUE	WHAT HAPPENS IF IT'S IGNORED
Loose boundaries between users	Any user could see or edit someone else's data just by guessing an ID number.
No control over request volume	One misbehaving script, or someone trying to guess a password, could overwhelm the whole service.
Documentation that's vague or out of date	Anyone trying to integrate with the API wastes time guessing what a field means or why a request failed.
No plan for changing the API later	The moment something needs to change, every existing user of the API breaks with no warning.
No visibility into what's happening	When something does go wrong, there's no record to figure out why.

TaskFlow API was built to address all five of these directly, in one connected system, rather than bolting on fixes later. Each decision below exists because of one of these problems, not as an afterthought.

03 Goals and Non-Goals

It's just as important to be clear about what this project intentionally does not try to do. Scope discipline is part of the design here, not a limitation of time.

What this project set out to do

- Let people securely create and manage their own projects and tasks
- Stop abuse of the system through sensible limits on how often someone can call the API
- Make the API genuinely easy to understand for any developer picking it up cold
- Allow the API to evolve later without breaking whoever is already using it
- Give non-technical people a way to actually use the product, not just read about it
- Show real production habits: containers, automated testing, structured logs

What it deliberately leaves out, for now

- Teams or shared organizations. Right now, every project belongs to exactly one person.
- Live, real-time collaboration, like seeing a teammate's edits appear instantly
- Any kind of billing or payments
- A native mobile app. The web dashboard is the only interface today.
- A permanent, publicly hosted version. Right now it's built to run locally (see the "What's Next" section for the plan to change that).

04 Who This Is For

Four fairly different kinds of people might end up looking at this project, and each one needs something different from it.

PERSON	WHAT THEY'RE ACTUALLY LOOKING FOR
A developer who wants to build on the API	Clear docs, predictable error messages, and confidence that the API won't quietly change and break their integration.
Someone with no technical background trying it out	The web dashboard. Sign up, create a project, add a task, done. No API knowledge required at all.
An engineer or hiring manager reviewing the code	Proof of judgment, not just working code. Why JWTs instead of sessions, why the URL includes a version number, why rate limits differ across endpoints.
The person who built it	A project they can explain confidently in an interview or a conversation, start to finish, not just something that happens to run.

05 What the Product Does

5.1 Signing up and logging in

Anyone can register with an email and a password. Passwords are never stored as plain text, they're hashed first, so even if the database were somehow exposed, the actual passwords wouldn't be readable. Logging in returns a token (a JWT) that proves who you are on every request after that, without the server having to remember a separate login session for you.

Every project and task belongs to exactly one user, and the API enforces that quietly but firmly: if you try to access someone else's project, you get a "not found" response rather than a "not allowed" one. That's intentional. It means the API never even confirms that another user's data exists, which closes off a common way attackers probe for information.

5.2 Projects and tasks

The core of the product is simple on purpose: you can create, view, edit, and delete both projects and tasks. Every task belongs to a project, and the API checks that the project is actually yours before letting you add anything to it. Tasks move through a simple lifecycle, from "To Do" to "In Progress" to "Completed."

5.3 Browsing lists without drowning in data

If you have hundreds of tasks, you shouldn't get all of them back in one giant response. Lists are paginated, meaning you get a page at a time, with a cap on how large a page can be so nobody can request an unreasonable amount of data in one go. Tasks can also be filtered, for example by status or by which project they belong to, so a client application only pulls what it actually needs.

5.4 Keeping the system from being overwhelmed

This is one of the two features this project is really built around. Not every part of the API carries the same risk, so not every part is limited the same way.

TYPE OF REQUEST	LIMIT	MEASURED PER	REASON
Logging in or registering	5 per minute	IP address	These are the endpoints someone would target to guess passwords, so they're the strictest.
Creating, editing, or deleting data	30 per minute	Logged-in user	Generous enough for normal use, tight enough to stop accidental or intentional flooding.
Reading data	100 per minute	Logged-in user	Reading is low-risk and should feel unrestricted in everyday use.

These limits are tracked in Redis, a shared, fast data store, so they hold up correctly even if the API is eventually running as multiple copies at once rather than a single process. If someone goes over their limit, they get a clear "too many requests" response, and the API tells them exactly how many requests they have left and when the limit resets, right in the response headers. The web dashboard even shows this live, so you can watch the limits in action instead of just reading about them.

5.5 Letting the API change without breaking things

This is the second core feature. Every endpoint lives under a version in its address, like `/api/v1/`, rather than the version being hidden in a header somewhere. That choice was made on purpose: a version number in the URL is something anyone can see, test, and understand immediately, whether they're reading documentation, pasting a link into a browser, or writing a quick script. The project also already has a way to mark an endpoint as deprecated and tell people exactly when it will be retired, even though there's no second version yet. Building that mechanism early means a future change doesn't have to be rushed or break anyone by surprise.

5.6 Documentation someone can actually use

Every endpoint has interactive documentation built in (available at `/docs` and `/redoc`), and it's been written with real explanations and examples rather than left as generic auto-generated text. There's also a separate design document that walks through the reasoning behind the bigger decisions (why JWTs, why this versioning approach, why these rate limits), aimed at anyone trying to understand the thinking, not just the mechanics. For people who prefer testing APIs in Postman rather than a browser, a ready-to-import collection is included too.

5.7 A dashboard for people who don't want to touch an API directly

A browser-based dashboard sits on top of the API so that someone with no technical background can use the product without ever seeing a line of code. You sign up, sign in, create a project, add some tasks, and change their status, all through a normal-looking web page. There's also a small panel that shows what's happening behind the scenes in real time, like current rate limits and system health, which turns things that are normally invisible into something anyone can actually watch happen.

5.8 Keeping a record of what happens

Every request the API handles gets logged in a structured, consistent format (which method was used, which path, what status code came back, how long it took), instead of scattered, unstructured print statements. This is the foundation for real monitoring later on. There's already a documented plan for what metrics would matter most in production (how much traffic is coming in, how often things fail, how slow the slowest requests are), even though a full metrics dashboard hasn't been built yet.

5.9 Making sure it actually works

The project has an automated test suite covering the important behavior: creating and editing data, logging in and out, pagination edge cases, rate limits actually kicking in, and one user never being able to see another user's data. As of this writing, that suite covers 99% of the codebase, and the automated pipeline that runs on every change refuses to let coverage drop below 95%. Those tests run against real database and cache services in the pipeline, not simulated stand-ins, so what passes in testing reflects how the system actually behaves.

5.10 Ready to actually run somewhere

The production version of the app is built as a multi-stage container, meaning the tools needed to build it are kept separate from what actually runs, which keeps the final image smaller and reduces what could go wrong. It also runs as a regular, non-admin user inside its container rather than as root, and nothing sensitive (passwords, secret keys) is hardcoded anywhere. All of that is passed in through environment variables instead.

06 Quality Bar (Non-Functional Requirements)

Beyond the features themselves, a few standards were held consistently across the whole project. These are the things that don't show up as a button on a screen, but that decide whether the product can actually be trusted.

AREA	WHAT'S EXPECTED
Security	Passwords are hashed, every request is authenticated, and no endpoint reveals whether a resource belonging to someone else exists.
Reliability	Nothing gets merged into the main codebase unless the automated tests pass and coverage stays above the agreed threshold.
Performance	List endpoints are paginated so they can't be asked to return unbounded amounts of data, and the database has indexes on the columns that get filtered most often.
Maintainability	Database changes are tracked through proper migrations from day one, logs are structured, and the reasoning behind decisions is written down, not just implied by the code.
Usability	Someone with no technical background can complete the entire workflow (sign up, create a project, add and update a task) using only the web dashboard.

07 How It's Built (System Architecture)

At a high level, a request from a browser or a tool like curl reaches a proxy layer, gets forwarded into the FastAPI application, and from there the app checks rate limits against Redis and reads or writes data in PostgreSQL. During automated testing, a separate lightweight SQLite database is used instead, so tests run quickly without touching real infrastructure.

```
graph TD
    Client[Client Browser / cURL]
    Proxy[Nginx Proxy / API Gateway]
    FastAPI[FastAPI Application]
    RateLimiter[Redis Rate Limit Store]
    DB[(PostgreSQL Database)]
    TestDB[(SQLite Test DB)]

    Client -->|HTTP Request| Proxy
    Proxy -->|Forward /api/v1| FastAPI
    FastAPI -->|Check Token / IP Limits| RateLimiter
    FastAPI -->|Query / Mutate| DB
    FastAPI -.->|Test Context Run| TestDB
```

The stack, in plain terms: FastAPI (a Python web framework) handles the actual API logic. PostgreSQL stores the real data, with SQLAlchemy and Alembic managing the database structure and its changes over time. Pydantic checks that incoming data actually looks the way it's supposed to. SlowAPI, backed by Redis, handles rate limiting. GitHub Actions runs the automated tests on every change. The dashboard itself is a plain HTML, CSS, and JavaScript page, nothing more exotic than that.

08 How the API Behaves

A few consistent rules run through every endpoint, so that using one part of the API teaches you how to use the rest of it.

- **Errors always look the same shape.** Every error comes back as a simple message in the same format, so any application built on top of this API only has to handle one error shape, not a different one for every endpoint.
- **Status codes mean what they're supposed to mean.** Successfully creating something returns 201. Deleting something returns 204. A request that breaks a business rule (like a duplicate name) returns 400. Being logged out or unauthorized returns 401. A missing resource, or someone else's resource, returns 404. Bad input returns 422. Going over a rate limit returns 429.
- **URLs stay simple.** Rather than deeply nesting tasks inside projects inside users in the address itself, the API keeps paths flat and enforces ownership rules in the application logic instead. It's easier to read and easier to work with.

09 What Success Looks Like

Because this is a portfolio project rather than a live commercial product, success isn't measured in revenue or daily active users. It's measured in whether the thing actually holds up the way it claims to.

WHAT'S MEASURED	TARGET	HOW IT'S ACTUALLY CHECKED
Test coverage	At least 95 percent	Currently sitting at 99 percent, visible directly on the repository's CI badge.
Build health	Every change on the main branch passes	Shown live through the GitHub Actions status badge.
Time for a new developer to understand the API	Under 10 minutes	Should be achievable just from the interactive docs and the design document, without reading source code.
Time for a non-technical visitor to complete the core workflow	Under 5 minutes	Achievable through the dashboard alone, following the onboarding steps in the README.
Data leaking between users	Zero incidents	Directly tested. Attempts to access another user's data are confirmed to return "not found," not real data.

10 Known Limitations

Being upfront about what's not finished yet is part of doing this properly. None of the following are surprises. They're documented tradeoffs, made consciously given the current stage of the project.

> **There's no live, public version yet.** Right now, anyone who wants to try this has to clone the repository and run it locally rather than just clicking a link. Fixing this is the next planned step.

> **It only supports individual users, not teams.** There's no concept yet of sharing a project with someone else or working together on the same board.

> **Rate limiting happens inside the application itself, not in front of it.** That's fine at the current scale, but it's already documented as something to move to a dedicated gateway layer once traffic grows enough to justify it.

> **Heavier tasks still run synchronously.** Anything resource-intensive currently happens directly within a request instead of being handed off to a background worker. The plan to introduce that is already written down, it just hasn't been built yet.

11 What's Next

These aren't vague ideas, they come straight from the scaling plan already written into the project's own documentation.

1. **Move rate limiting out to a dedicated gateway** (something like Kong, Nginx, or Cloudflare), so the application itself isn't spending its own resources policing traffic.
2. **Add read replicas to the database.** Since most of the traffic on a task management tool is people reading their boards rather than writing to them, splitting reads onto separate database copies would ease pressure on the main one.
3. **Add a caching layer with Redis** for the data that gets read most often, so repeated requests don't have to hit the database every single time.
4. **Move slow or heavy work to background workers** (using something like Celery or ARQ), so things like sending an email or generating a report don't hold up a normal request.
5. **Actually deploy a public, live version.** This is a natural next step, especially since the interactive documentation already works as a self-contained demo on its own.

12 Glossary of Terms

A few terms come up throughout this document that assume some technical background. Here's what each one actually means.

TERM	WHAT IT ACTUALLY MEANS
REST API	A common, predictable way for two pieces of software to talk to each other over the internet, using simple actions like "get this," "create this," or "delete this."
JWT (JSON Web Token)	A small, secure piece of data that proves who a user is, without the server having to keep track of a separate login session for every person.
Rate limiting	A safeguard that limits how many requests one person or one address can send in a short window of time, so the system can't easily be overwhelmed or abused.
Versioning	Marking which version of the API someone is using directly in the web address, so changes to the API later don't unexpectedly break people who are still using an older version.
CI, or Continuous Integration	An automated process that runs the full test suite every time the code changes, to catch problems before they ever reach a real user.
Test coverage	Roughly, the percentage of the codebase that's actually exercised by automated tests. It's not a perfect measure, but it's a reasonable signal of how much of the system has been genuinely verified.